

Web Development Learning Guide

HTML, CSS & Typography Fundamentals

A Complete Reference Based on Your Project Files

1. HTML Basics

HTML (HyperText Markup Language) is the foundation of web pages. It provides structure and content.

1.1 Document Structure Tags

Tag	Purpose	Contains
<!DOCTYPE html>	Declaration. Always first line.	Tells browser: HTML5
<html> ... </html>	Root element wrapper	Entire page
<head> ... </head>	Metadata (not visible)	Title, links, styles
<body> ... </body>	Visible content	Text, images, links
<title> ... </title>	Browser tab text	Page title only

1.2 Important Meta Tags

<meta charset='UTF-8'> — Declares character encoding. Essential for special characters.

<meta name='viewport' content='width=device-width, initial-scale=1.0'> — Makes page mobile responsive.

<title>Page Title</title> — Shown in browser tab and bookmarks.

1.3 Heading Tags (h1 to h6)

Tag	Size/Importance	Best Use	Rules
<h1>	Largest/Most Important	Main page title	Use ONCE per page
<h2>	2nd Level	Section headings	Use multiple times
<h3>	3rd Level	Sub-sections	Nest under h2
<h4>	4th Level	Sub-sub-sections	Nest properly
<h5>, <h6>	5th & 6th Level	Rarely used	Avoid if possible

1.4 Text Formatting Tags

Tag	Visual Effect	Has Meaning?	When to Use	Note
	Bold text	No - styling	Rarely	Avoid this
	Bold text	YES - important	For important text	Preferred!
<i>	Italic text	No - styling	Rarely	Avoid this
	Italic text	YES - emphasis	For emphasis	Preferred!
<p>	Paragraph block	YES - semantic	All text blocks	Essential
<sup>	Superscript	YES - semantic	Exponents, ordinals	x²
<sub>	Subscript	YES - semantic	Chemistry (H2O)	H₂O

KEY CONCEPT: Semantic vs Visual Tags

Use `<strong>` instead of `<b>`, and `<em>` instead of `<i>`. Semantic tags improve accessibility and SEO. Screen readers understand `<strong>` means importance, and `<em>` means emphasis.

1.5 Special & Semantic Tags

`<blockquote cite='URL'>` — Long quote from external source. Browsers indent it.

`<abbr title='full text'>` — Abbreviation. Hover shows full expansion.

`<hr>` — Horizontal line divider. Self-closing (no closing tag).

`<br>` — Line break. Self-closing. Use sparingly.

`<nav>` — Semantic navigation container.

1.6 HTML Common Mistakes

Forgetting closing tags. Always close `<p>` with `</p>`.

Using `<b>` instead of `<strong>` for importance.

Multiple `<h1>` tags per page. Use only ONE h1 as main title.

Missing viewport meta tag. Add it for mobile responsiveness.

Poor heading structure. Nest logically (h1 to h2 to h3).

2. CSS Fundamentals

CSS (Cascading Style Sheets) controls visual appearance. It handles colors, fonts, spacing, and layout.

2.1 CSS Syntax

CSS selects HTML elements and applies styles. Format: selector { property: value; }

2.2 Common CSS Selectors

Selector	Syntax	Selects	Specificity
Element	p { }	All p tags	Low
Class	.classname { }	class="classname"	Medium
ID	#idname { }	id="idname"	Highest
Attribute	[type="text"] { }	Specific attributes	Medium
Descendant	nav ul { }	ul inside nav	Low

2.3 Typography Properties

Property	Values	Example
font-size	px, em, rem, %	font-size: 32px
font-weight	normal, bold, 100-900	font-weight: bold
font-style	normal, italic, oblique	font-style: italic
text-align	left, center, right, justify	text-align: center
color	color names, hex, rgb	color: #333333
line-height	number, px, em	line-height: 1.6
letter-spacing	px, em	letter-spacing: 2px

2.4 The CSS Box Model

Every element has: MARGIN (outside) - BORDER - PADDING (inside) - CONTENT

Property	What It Is	Purpose	Example
margin	Space outside border	Separation from others	margin: 20px
border	Line around element	Visual boundary	border: 2px solid black
padding	Space inside, before content	Internal spacing	padding: 15px
width/height	Element dimensions	Size control	width: 100px

2.5 Display Property (Most Important!)

Value	Behavior	Example Elements
block	Full width, new line	div, p, h1, nav

inline	Only needed width, flows	span, a, strong, em
inline-block	Flows like inline + respects size	buttons, custom boxes
flex	Flexible layout (Flexbox)	navigation, grids
grid	Two-dimensional layout	dashboards, galleries
none	Hidden from display	hidden elements

2.6 Interactive Styles with Pseudo-classes

:hover — Triggered on mouseover. Example: `a:hover { color: red; }`

:focus — Triggered when element focused. Example: `input:focus { background-color: aqua; }`

:active — Triggered on click. Example: `button:active { opacity: 0.8; }`

2.7 CSS Common Mistakes

Using hardcoded pixels everywhere. Use flexible units like %, em, or rem.

Using absolute positioning for layouts. Use flexbox or grid instead.

Ignoring mobile responsiveness. Always test on mobile devices.

Inconsistent color palette. Create and follow a color scheme.

3. Typography Fundamentals

Typography is the art of arranging text beautifully and readably.

3.1 Font Weight

Weight	CSS Value	Use For	Visual Effect
Thin	100	Display, large titles	Very light, elegant
Light	300	Secondary content	Light weight
Normal	400 or normal	Body text	Standard weight
Medium	500	Emphasis, highlights	Slightly heavier
Bold	700 or bold	Headers, important	Heavy, prominent
Black	900	Strong emphasis	Very heavy, bold

3.2 Font Style

Normal (regular) — Standard upright text. Default for body content.

Italic — Curved/slanted text. Use for emphasis, quotes, citations.

Oblique — Similar to italic but more subtle. Available on some fonts.

3.3 Font Stretch (Width)

Condensed — Narrower letters. Fits more text in less space.

Normal/Regular — Standard letter width. Default for all text.

Extended — Wider letters. Use for emphasis or dramatic headlines.

3.4 Typography Best Practices

Use 2-3 font families maximum. Too many fonts creates chaos.

Maintain consistent font sizes. Create a scale: h1=64px, h2=48px, p=16px.

Use adequate line-height. Aim for 1.5-1.8 for readability.

Ensure high contrast. Dark text on light background (or vice versa).

Use web-safe fonts or Google Fonts. Ensures consistent display.

4. Web Development Best Practices

4.1 Code Organization

Use single index.html file (consolidate all versions)

Create separate styles.css file and link properly

Use meaningful class and ID names (.navigation, #main-content, not .box1)

Add comments explaining complex code sections

4.2 Responsive Design

Always include viewport meta tag for mobile devices

Use flexible units (% , em, rem) instead of fixed pixels

Use flexbox or CSS Grid, not absolute positioning

Test on mobile, tablet, and desktop screens

4.3 Accessibility (a11y)

Use semantic HTML: <nav>, <main>, <header>, <footer>

Add alt text to images for screen readers

Use <label> for form inputs

Ensure color contrast meets WCAG standards

Use proper heading hierarchy (h1 to h2 to h3)

4.4 Performance

Optimize images before uploading (compress PNG/JPG)

Minify CSS and JavaScript in production

Load external resources efficiently

Limit web fonts (slower=bad, use 1-2 fonts max)

5. Pro Tips for Success

Practice Consistently

Build something every day. Start with simple projects, progress to complex ones.

Learn by Reading Others' Code

View source on websites you like. Use browser developer tools (F12) to inspect and experiment.

Don't Memorize, Understand

Focus on concepts, not memorization. Use references (Google, MDN) when needed.

Join Web Dev Communities

Share your work, get feedback, learn from others. Collaborate on projects.

Validate Your Code

Use W3C validators to check HTML and CSS. Fix errors for better compatibility.

Your Learning Journey

Step 1: Master Basics

Create simple web pages with clean HTML. Use semantic tags. Practice proper structure.

Step 2: Style Everything

Apply CSS to create beautiful layouts. Learn flexbox. Master responsive design.

Step 3: Refine & Polish

Add animations, transitions, hover effects. Test across devices. Optimize accessibility.

Step 4: Build Real Projects

Create a portfolio. Build a personal website. Contribute to open source.

Happy Coding! Keep Building. Keep Learning. ■